



Capítulo 2

Sintaxis y Diseño de Suites de Prueba

Objetivos:

- Describir la relación estructural entre test cases, keywords y bibliotecas dentro de una suite de Robot Framework.
- Explicar las funcionalidades principales de las bibliotecas BuiltIn, Collections y OperatingSystem con ejemplos concretos.
- Interpretar la diferencia entre keywords de alto nivel y keywords técnicas para comprender cómo se organizan las pruebas.
- Los test cases describen el qué verificar; las keywords describen el cómo, y las bibliotecas proveen el vocabulario técnico listo para usar.

2.1: Test Cases, Keywords y Bibliotecas: BuiltIn, Collections, OperatingSystem

- Introducción

Un test case dice QUÉ se prueba. Una keyword dice CÓMO se hace.

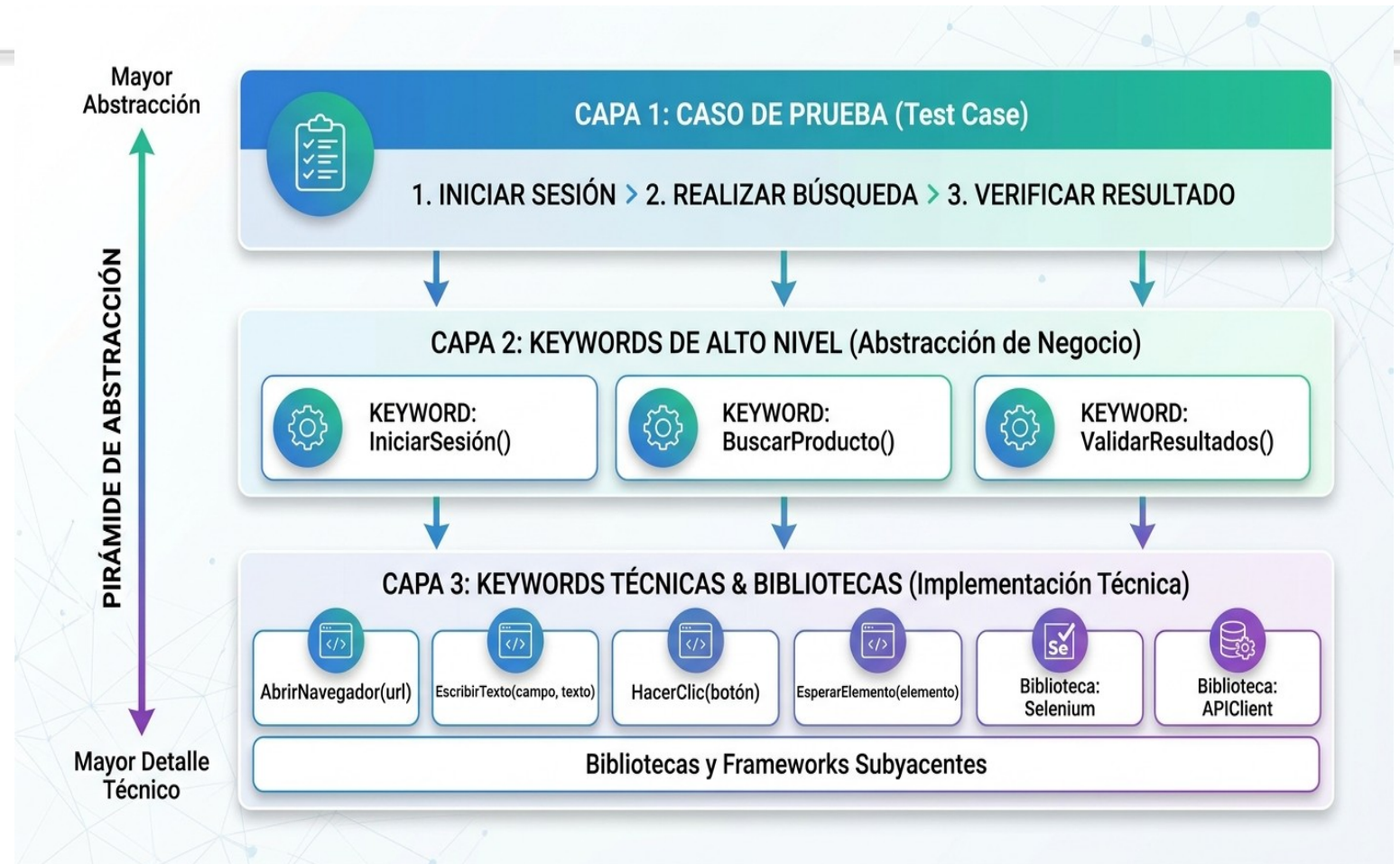
Las librerías (BuiltIn, Collections, OperatingSystem) ponen las keywords ya construidas a tu disposición.

Test Cases y Keywords: La Unidad Fundamental

- ▶ Un test case es una unidad de prueba con nombre que contiene una secuencia de pasos.
- ▶ Cada paso es una invocación a una keyword: nativa, de biblioteca o definida por el equipo.
- ▶ Las keywords encapsulan acciones o verificaciones con nombres legibles en lenguaje natural.
- ▶ Pueden recibir argumentos, devolver valores y anidarse unas dentro de otras.
- ▶ El test case describe el qué verificar; la keyword describe el cómo hacerlo.
- ▶ La composición de keywords permite construir pruebas complejas desde bloques simples.

Capas de Abstracción en Robot Framework

- ▶ Test case: nivel de negocio, legible por stakeholders.
- ▶ Keyword de negocio: orquesta pasos del flujo funcional.
- ▶ Keyword técnica: interactúa con el sistema real.
- ▶ Separar capas facilita mantenimiento y colaboración.



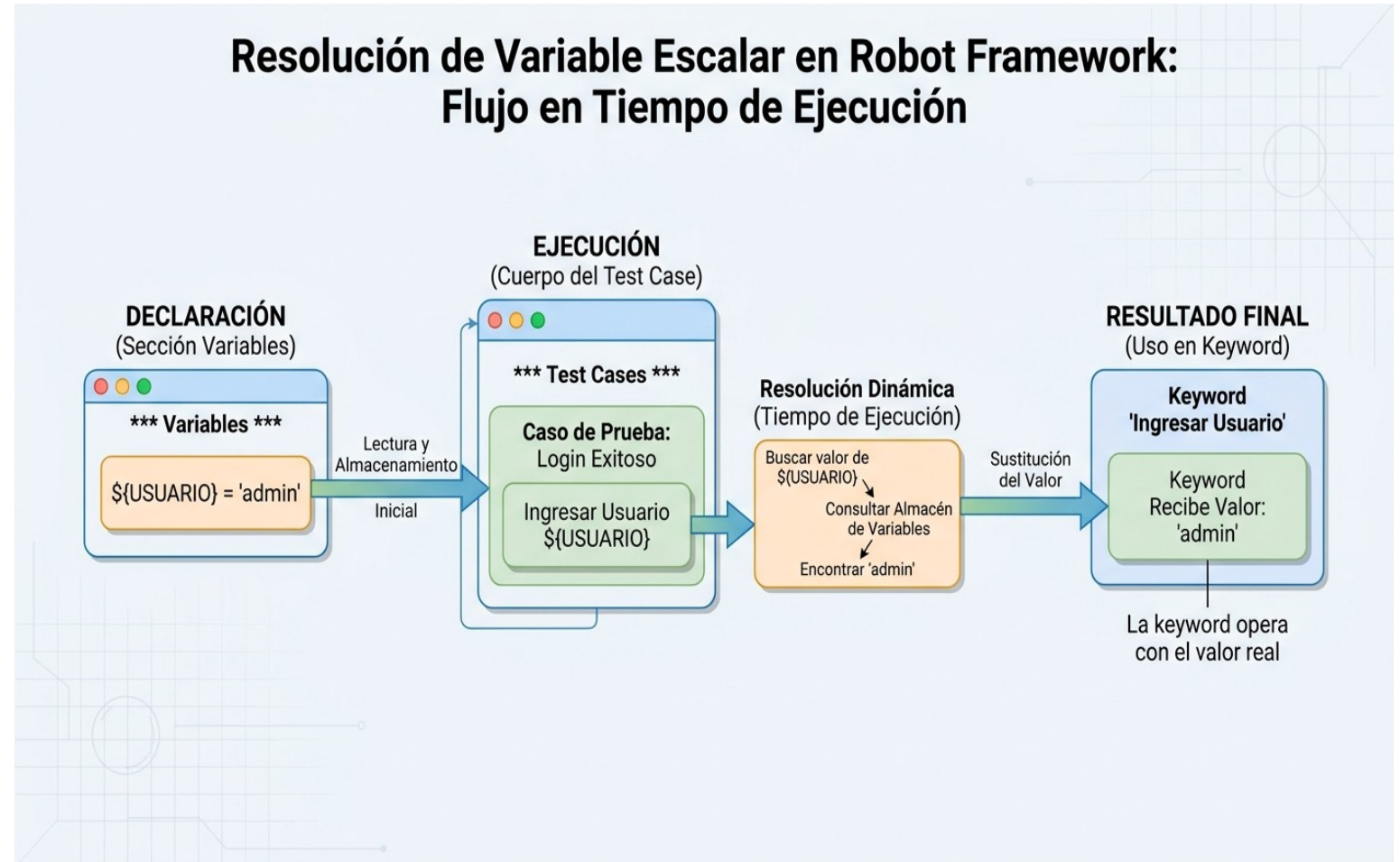
2.2: Variables: Escalares, Listas y Diccionarios

- Introducción

Tres tipos de variables en Robot Framework: escalares (`${...}`) para un solo valor, listas (`@{...}`) para varios valores, y diccionarios (`&{...}`) para pares clave-valor.

Variables Escalares (\${var})

- ▶ Almacenan un único valor: texto, número, booleano u objeto Python.
- ▶ Nombres insensibles a mayúsculas y espacios.
- ▶ Centraliza configuración: URLs, credenciales, timeouts.
- ▶ Precedencia: *** Variables *** < Resource < CLI < test case.

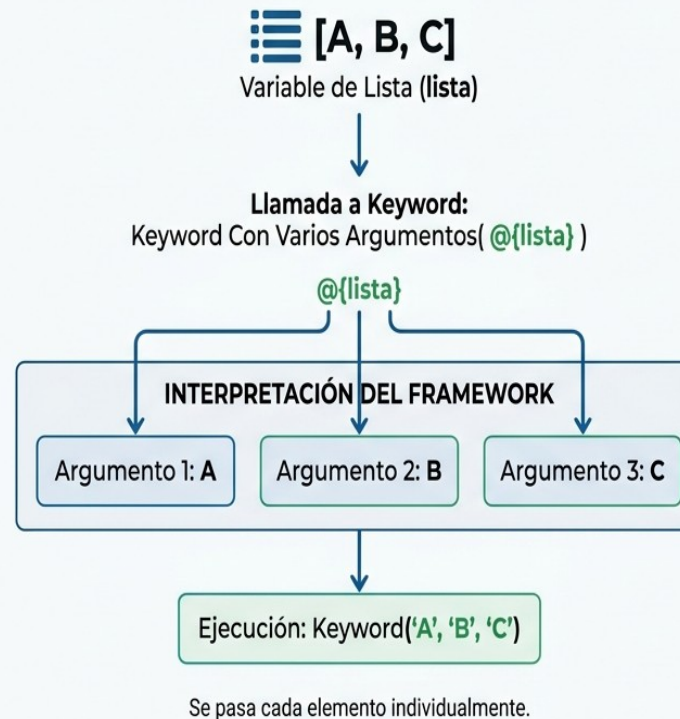


Variables de Lista (@{list})

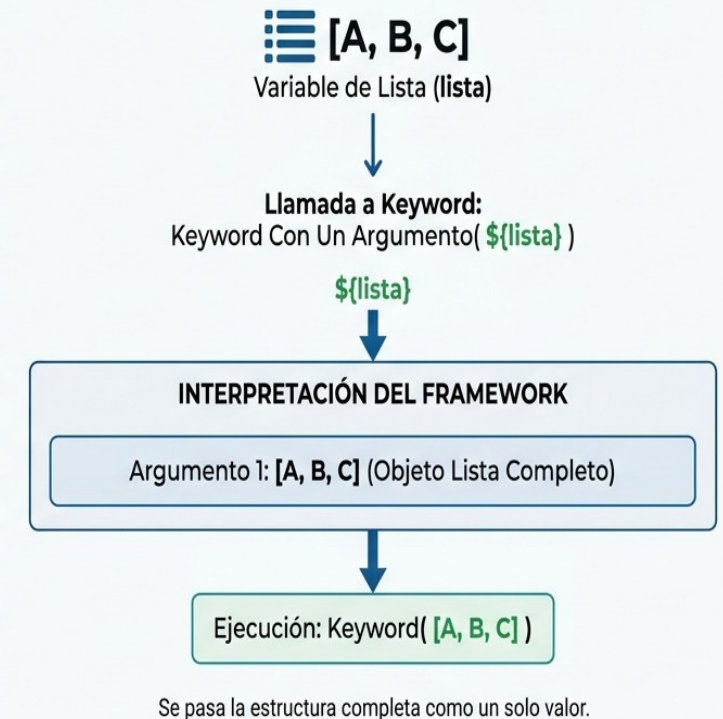
- ▶ @{list} expande elementos como argumentos posicionales separados.
- ▶ \${lista} trata la misma variable como un único objeto Python.
- ▶ Acceso por índice con \${lista}[0].
- ▶ Ideales para iterar con bucles FOR sobre colecciones.

Diagrama Comparativo: Expansión de Lista (@{lista}) vs. Objeto Lista (\${lista})

EXPANSIÓN EN MÚLTIPLES ARGUMENTOS (@{lista})



OBJETO ÚNICO (\${lista})



Variables de Diccionario (&{dict})

- ▶ Almacenan pares clave-valor; equivalente a `**kwargs` en Python.
- ▶ `&{dict}` expande los pares como argumentos con nombre al llamar keywords.
- ▶ Acceso a campos individuales: `{dict}[clave]`.
- ▶ Perfectos para headers HTTP, payloads JSON y configuración agrupada.

COMPARACIÓN DE TIPOS DE DATOS: ESCALAR, LISTA, DICCIONARIO

	 ESCALAR	 LISTA	 DICCIONARIO
 Notación	42, 'hola', 3.14	[1, 2, 3], ['a', 'b', 'c']	{'clave': 'valor', 'id': 101}
 Tipo de Dato Almacenado	Valor único (entero, texto, booleano)	Secuencia ordenada de elementos	Colección no ordenada de pares clave-valor
 Caso de Uso Típico	Contadores, flags, datos simples	Series temporales, registros de eventos, inventarios	Configuración, perfiles de usuario, búsqueda rápida
 Forma de Acceso a Elementos Individuales	Directo por referencia (ej. <code>nombre_variable</code>)	Por índice numérico (ej. <code>lista[0]</code>)	Por clave única (ej. <code>diccionario['clave']</code>)

2.3: Archivos Resource: Separación de Keywords y Parametrización de Datos

- Introducción

Cuando una suite crece, conviene separar las keywords en un archivo Resource — así cualquier suite puede reutilizarlas sin duplicar código.

Archivos Resource: Concepto y Propósito

- ▶ Un archivo Resource (.resource o .robot) centraliza keywords, variables e importaciones reutilizables.
- ▶ No puede contener test cases ni tasks; su rol es proveer bloques de construcción a otras suites.
- ▶ Sigue el principio de responsabilidad única: cada archivo cubre un dominio concreto.
- ▶ Reduce el acoplamiento entre componentes y facilita la búsqueda y corrección de errores.
- ▶ Actúa como biblioteca compartida: un cambio en un solo lugar beneficia a todas las suites.

Secciones Permitidas en un Archivo Resource

- ▶ **Settings** : importa librerías (Library) u otros archivos Resource (Resource).
- ▶ **Variables** : declara variables escalares, listas y diccionarios compartidos.
- ▶ **Keywords** : define keywords reutilizables con pasos, argumentos y documentación.
- ▶ **Test Cases** NO está permitida; incluirla genera un error de parseo.
- ▶ Se importa con la directiva Resource en **Settings** del archivo consumidor.
- ▶ Usar rutas relativas hace el proyecto portable entre distintos entornos.

2.4: Tags, Setup/Teardown y Organización Jerárquica de Suites

- Introducción

Dos herramientas para ordenar suites grandes: los tags (para filtrar qué se ejecuta) y Setup/Teardown (para preparar y limpiar antes/después de los tests).

Tags en Robot Framework

- ▶ Los tags son etiquetas de texto libre para clasificar, filtrar y reportar test cases.
- ▶ Un test case puede tener múltiples tags simultáneamente (smoke, regresión, módulo, etc.).
- ▶ Se declaran en Test Tags (nivel suite) o [Tags] (nivel test case individual).
- ▶ Los tags de suite se combinan automáticamente con los tags del test case.
- ▶ Desde RF6, Test Tags reemplaza a Force Tags y Default Tags de versiones anteriores.
- ▶ Permiten agrupar pruebas por funcionalidad, prioridad, entorno o sprint.

Setup y Teardown: Concepto

- ▶ Setup se ejecuta automáticamente ANTES de un test case o suite.
- ▶ Teardown se ejecuta automáticamente DESPUÉS de un test case o suite.
- ▶ Garantizan que el entorno esté correcto y que los recursos queden liberados.
- ▶ El Teardown SIEMPRE se ejecuta, incluso si el test case falla.
- ▶ Separan la lógica de preparación de la lógica de prueba, mejorando legibilidad.
- ▶ Analogía: preparar la mesa antes de la reunión (Setup) y recoger al terminar (Teardown).

Setup y Teardown: Cuatro Niveles de Ejecución

Nivel	Keyword de configuración	Cuándo se ejecuta
Test Case	[Setup]/[Teardown]	Antes/después de cada test case individual
Suite (archivo)	Suite Setup/Suite Teardown	Al inicio/final del archivo.robot
Suite (directorio)	Suite Setup/Suite Teardownen__init__.robot	Al inicio/final del directorio completo

Organización Jerárquica de Suites

- ▶ Una suite puede ser un archivo .robot individual o un directorio completo.
- ▶ Anidar suites en directorios refleja la estructura del sistema bajo prueba.
- ▶ La jerarquía afecta reportes, herencia de settings y ejecución selectiva.
- ▶ `__init__.robot` en un directorio define Suite Setup/Teardown para todos sus archivos.
- ▶ Robot Framework nombra suites automáticamente: `crear_factura.robot` → Crear Factura.
- ▶ Se pueden ejecutar directorios completos, subdirectorios o archivos individuales.

Resumen del capítulo

- Los **test cases** describen el *qué* verificar; las **keywords** describen el *cómo*, y las **bibliotecas** proveen el vocabulario técnico listo para usar
- La separación en capas de abstracción (test case → keyword de negocio → keyword técnica) es la práctica central para suites mantenibles
- **BuiltIn** está siempre disponible sin importación y cubre verificaciones, logging, variables y control de ejecución
- **Collections** se importa explícitamente y es esencial para trabajar con listas y diccionarios, estructuras omnipresentes en datos de prueba y respuestas de API
- **OperatingSystem** permite interactuar con el sistema de archivos y el entorno, siendo clave para flujos RPA y generación de evidencias



Resumen del capítulo

- Las tres bibliotecas forman un conjunto complementario que cubre la mayoría de necesidades en pruebas funcionales básicas
- Las variables **escalares** (`{var}`) almacenan un único valor y son el tipo más común; se usan para configuración, resultados de keywords y valores reutilizables.
- Las variables de **lista** (`@{list}`) agrupan colecciones ordenadas; al usarse con `@{}` se expanden como argumentos múltiples, mientras que con `${}` se tratan como un objeto único.
- Las variables de **diccionario** (`{dict}`) organizan datos en pares clave-valor; al usarse con `{}` se expanden como argumentos con nombre, lo que facilita la integración con APIs y librerías Python.
- La **precedencia** de variables permite sobrescribir valores desde la línea de comandos, lo que es fundamental para ejecutar las mismas suites en distintos entornos sin modificar el código.



Práctica 3: Suite estructurada con keywords reutilizables y archivo Resource

Objetivo:

Diseñar una suite con keywords reutilizables, importar un archivo Resource y validar datos con variables compartidas.

Guía completa: [robfram-guide/Capitulo02/Practica3.md](#)



Práctica 4: Parametrización con Setup/Teardown y filtrado por tags

Objetivo:

Implementar Suite/Test Setup y Teardown, y ejecutar subconjuntos de tests usando tags de inclusión/exclusión.

Guía completa: [robfram-guide/Capitulo02/Practica4.md](#)



Referencias Bibliográficas

Capítulo 2: Sintaxis y Diseño de Suites de Prueba

- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>
- ▶ <https://robotframework.org/robotframework/latest/libraries/BuiltIn.html>
- ▶ <https://robotframework.org/robotframework/latest/libraries/Collections.html>
- ▶ <https://robotframework.org/robotframework/latest/libraries/OperatingSystem.html>
- ▶ <https://github.com/robotframework/robotframework>